

Ad Hoc Teamwork in a Zombie Survival Grid World

Francisco Nascimento
ist1106559

Miguel Baptista
ist1107312

Daniel Bartolomeu
ist1106182



Figure 1: Live simulation of the zombie survival grid world

Abstract

We study ad hoc teamwork in a multi-agent zombie survival scenario implemented as a 30×30 grid world. Three survivor agents (Scout, Defender, and Support) must cooperate to reach one of two safe zones while evading and fighting zombies. An *adaptive agent* replaces one or all fixed-role teammates without prior coordination, learning to select among three role-behaviours at each timestep via tabular reinforcement learning (Q-Learning and SARSA). Experiments over 500 test episodes compare multiple replacement scenarios and learning configurations. The best success-survival trade-off was obtained with SARSA and ϵ decay 0.995, where the fully adaptive team reached a success rate of 0.936 and an average of 2.64 surviving agents.

ACM Reference Format:

Francisco Nascimento, Miguel Baptista, and Daniel Bartolomeu. 2026. Ad Hoc Teamwork in a Zombie Survival Grid World. In *Proceedings of . ACM*, New York, NY, USA, 5 pages.

1 Introduction

Ad hoc teamwork [1] requires an agent to cooperate with teammates whose strategies are unknown and cannot be agreed upon in advance. This is a fundamental challenge in multi-agent systems, with applications ranging from search-and-rescue robotics to autonomous driving.

We model this problem as a zombie survival task. Three survivor agents spawn in the bottom-left corner of a 30×30 grid and must navigate to one of two randomly placed safe zones while 12 zombies actively pursue them. The environment contains 15 obstacles and operates under partial observability: the safe zone locations are unknown until discovered by exploration or scanning.

Each fixed-role agent follows a hand-coded behaviour: the **Scout** explores and scans for safe zones; the **Defender** intercepts and attacks threats; the **Support** heals injured teammates. The **Adaptive** agent observes a compact state and selects which role behaviour to execute at each step, learning this policy through repeated interaction.

Our contributions are: (i) a reproducible multi-agent grid-world benchmark for ad hoc teamwork; (ii) a state and reward design that enables tabular RL to solve the role-selection problem; and (iii) an empirical comparison of Q-Learning vs. SARSA across six hyperparameter settings.

2 Related Work

The concept of ad hoc teamwork was formally introduced by Stone et al. [1], who challenged the AI community to build agents capable of collaborating with previously unknown teammates without pre-coordination. Their evaluation paradigm defines an ad hoc team agent as one that must perform well when inserted into arbitrary subsets of a fixed teammate pool, precisely the setting we adopt, where the adaptive agent is evaluated against all combinations of fixed-role teammates.

Stone et al. also decompose the challenge along three axes: *team-mate characteristics* (action and sensing capabilities), *team characteristics* (size, homogeneity), and *task characteristics* (goal, time horizon, coordination tightness) [1]. Our scenario instantiates a constrained point in this space: teammates are fixed-policy agents with known role behaviours, the team is small and heterogeneous (Scout, Defender, Support), and the task requires tight real-time coordination under partial observability.

Our role-switching formulation is closest in spirit to behaviour selection approaches, where a meta-policy selects among a fixed library of sub-behaviours [1]. By reusing the existing Scout, Defender, and Support `step()` methods as macro-actions, we keep the

action space tractable (3 actions, 206–224 reachable states) while still allowing the agent to adapt its strategy to the current team composition, addressing the ad hoc team challenge at the role level rather than the motion-primitive level.

Reinforcement learning has also been widely studied as a mechanism for coordination in cooperative multi-agent systems. Claus and Boutilier [4] showed that RL agents can learn coordinated behaviours in shared-reward settings, while Panait and Luke [5] survey broader challenges in cooperative multi-agent learning. Our work differs from full multi-agent reinforcement learning because the adaptive agent does not learn primitive actions directly. Instead, it learns a meta-policy over existing role behaviours, making the problem smaller and more interpretable while still testing adaptation to different teammate compositions.

3 Environment and Agents

3.1 Environment

The simulation runs on a 30×30 MultiGrid using the Mesa framework [2]. Each episode ends when all survivors reach a safe zone (*success*), all survivors die (*failure*), or 300 steps elapse (*timeout*). Two 2×2 safe-zone clusters are randomly placed at least 15 cells from the spawn corner. Zombies cannot enter safe zones, and obstacles block movement for all agents.

Agent perception in the grid is limited and measured using Manhattan distance. By default, all survivors share a standard vision range of 5 cells, allowing them to spot nearby safe zones or immediate threats. The Scout, however, provides a crucial tactical advantage: an active scan range of 9 cells, making it the only team member capable of long-range discovery. Meanwhile, zombies operate with a vision range of 6 cells. This creates a dangerous asymmetric dynamic, as zombies can detect and begin pursuing survivors from just outside the human line of sight.

3.2 Fixed-Role Agents

All fixed-role survivors inherit from the same base class and share core utilities for movement, safe-zone discovery, pathfinding via Breadth-First Search (BFS), and communication of discovered safe-zone cells. Role-specific behaviours then use these utilities differently: the Scout focuses on exploration, the Defender on combat and protection, and the Support on healing and team survival.

Scout. The Scout acts as the team’s exploration and information-gathering agent. It is the only fixed-role survivor with an active long-range scan, which allows it to discover safe zones before they are visible to the rest of the team. When no safe zone is known, it explores the map using exploration targets; once a safe zone has been discovered, it attempts to guide the team towards it while avoiding nearby zombies and preventing the group from becoming too spread out.

Defender. The Defender focuses on combat and protection. It attacks zombies that are adjacent to it and moves to intercept zombies that threaten nearby teammates. When no immediate threat is present, it follows the team’s progress towards a known safe zone, helps stay close to vulnerable teammates, and otherwise supports exploration by following the Scout or staying near the Support.

Support. Support. Dedicated to team survival and cohesion, this agent primarily heals adjacent injured teammates. Under threat, it

avoids direct confrontation by seeking protection from the Defender or retreating. When safe, it actively approaches wounded allies to assist them, or advances towards the discovered safe zone. In the absence of a clear objective, it shadows the Defender or Scout to aid in coordination, resorting to independent exploration only as a last resort.

3.3 Adaptive Agent

The adaptive agent inherits from the base survivor class and at each step chooses one of three *actions*: scout, defender, or support. Executing an action runs the corresponding role’s `step()` method on the adaptive agent’s own body, reusing all perception and movement utilities.

State representation: The state is a 6-tuple: health bucket (low/medium/ high), zombie distance bucket (danger/near/far/none), nearby zombie count (none/few/many), safe-zone phase (unknown/near/medium/ far), injured teammate in heal range (boolean), team too far (boolean, threshold 5 cells). The theoretical state space has $3 \times 4 \times 3 \times 4 \times 2 \times 2 = 576$ combinations, but most are unreachable in practice, for example, a ‘danger’ zombie distance with ‘none’ zombie count is impossible. Our experiments observe 206–224 reachable states, confirming that tabular RL is tractable here without approximation

Reward function: The adaptive agent uses a shaped reward function designed to encourage safe-zone discovery, team protection, healing, and mission completion, while penalising unsafe or inefficient behaviour. Table 1 summarises the reward components.

Table 1: Reward function components.

Event / Condition	Reward
Step penalty	−0.1
Move closer to discovered safe zone	+1.0
Move away from discovered safe zone	−0.8
Safe-zone discovery	+3.0
Successful scan discovery	+1.5
Attack event	+4.0
Heal event	+5.0
Reach safe zone	+5.0
Zombies away from teammates	+0.5
Lose health	−2.0
Teammate death	−12.0
Own death	−20.0
Mission success	+20.0 + 3×survivors

Learning algorithms. We implement tabular Q-Learning (off-policy) and SARSA (on-policy). The Q-table is initialised to zero. At each step the agent selects action a via ϵ -greedy, executes it, observes reward r and next state s' , and updates:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \hat{Q}(s') - Q(s, a)] \quad (1)$$

where $\hat{Q}(s') = \max_{a'} Q(s', a')$ for Q-Learning and $\hat{Q}(s') = Q(s', a')$ (chosen next action) for SARSA. We use $\alpha=0.2$, $\gamma=0.9$.

4 Experiments

4.1 Setup

The agent is trained for 1500 episodes across four team configurations in round-robin: *adaptive_replaces_scout*, *adaptive_replaces_defender*, *adaptive_replaces_support*, and *adaptive_adaptive_adaptive*. We compare six RL configurations (Table 2) across five team modes (4.2), each tested for 500 episodes with $\epsilon=0$.

Table 2: RL training configurations compared.

Algorithm	ϵ_0	Decay
Q-Learning	1.0	0.999
SARSA	1.0	0.999
Q-Learning	1.0	0.995
SARSA	1.0	0.995
Q-Learning	0.25	1.0 (none)
SARSA	0.25	1.0 (none)

4.2 Team Configurations

Five configurations are evaluated:

- (1) **Baseline:** Scout + Defender + Support (no adaptive agent).
- (2) **Adapts Scout:** Adaptive replaces Scout.
- (3) **Adapts Defender:** Adaptive replaces Defender.
- (4) **Adapts Support:** Adaptive replaces Support.
- (5) **Adapts (All):** Three adaptive agents.

5 Results and Discussion

Table 3: Test results across 500 episodes (SARSA, $\epsilon_0=1.0$, decay 0.995). Success rate and average survivors.

Configuration	Success	Avg. Survivors
Baseline	0.816	2.17
Adapts Scout	0.868	2.59
Adapts Defender	0.824	1.94
Adapts Support	0.822	1.73
Adapts (All)	0.936	2.64

Overall Performance: The *Adapts (All)* configuration consistently achieves the highest success rate (0.922–0.946 across all RL variants), well above the baseline of 0.816 (Table 3, Figure 5). Replacing the Scout with an adaptive agent also yields consistent gains (0.840–0.892), confirming that role-switching is most beneficial when the agent substitutes the most strategically demanding role.

Hardest Replacements (Defender and Support): When the adaptive agent replaces the Defender or Support, the success rate remains comparable to the baseline, but the average number of survivors drops visibly (to 1.94 and 1.73, down from 2.17). This is expected because both roles require highly reactive behaviours (immediate attack or heal) triggered by tight local conditions. The adaptive agent learns to approximate them but can miss the one-step reaction window due to the coarser granularity of role-level switching.

Learning Dynamics: Figure 2 illustrates the consolidated training progression. The fully adaptive team (Figure 3a) exhibits a robust climb: success starts at 0.6 around episode 50 and stabilises above the baseline by episode 200, with survivor counts rising from 1.0 to 2.2+ within 300 episodes. In sharp contrast, replacing the Defender (Figure 3b) yields a highly volatile curve that struggles to consistently surpass the baseline throughout training, reflecting the high cost of missing a defensive action. Test performance at $\epsilon=0$ does eventually match the baseline, showing that the learned policy is sound even if the training signal is noisy.

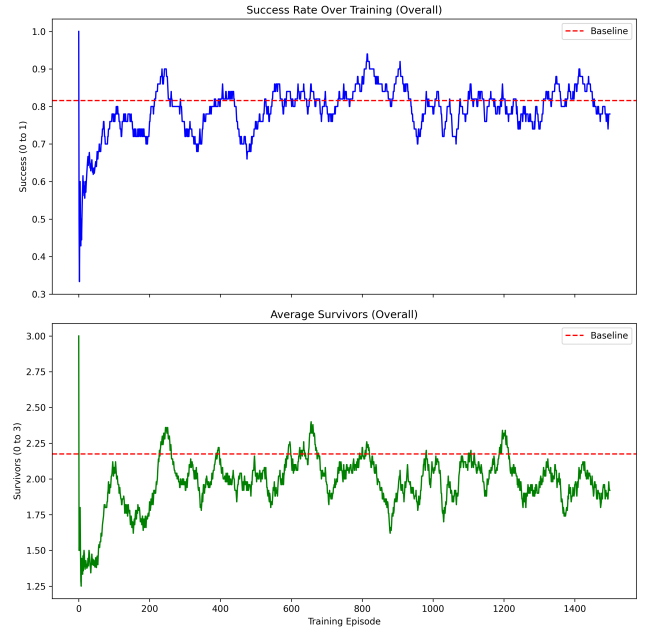
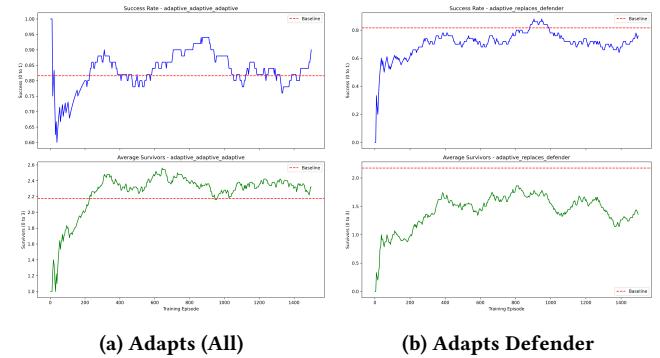


Figure 2: Learning curves (success rate and avg. survivors) over 1500 training episodes.



(a) Adapts (All)

(b) Adapts Defender

Figure 3: Specific learning curves.

Role Selection: Figure 4 reveals the core of the learned ad hoc policy. Across all configurations the adaptive agent predominantly selects scout (≈ 41 – 61%). This is strategically sound: safe-zone discovery is the primary mission bottleneck; without a known goal, neither attack nor heal actions can advance the team. Notably, when replacing the Support the agent dynamically increases its support usage ($\approx 38\%$) relative to the other scenarios, demonstrating successful contextual adaptation to the team’s structural deficit.

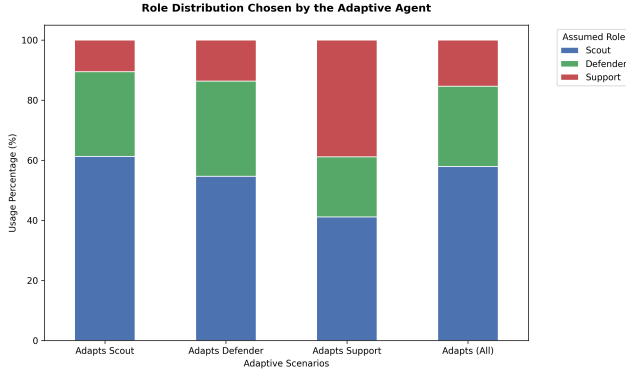


Figure 4: Distribution of role-behaviours chosen by the adaptive agent across test scenarios.

Q-Learning vs. SARSA: Both algorithms perform comparably across all team configurations (Figure 5). Although SARSA with decay 0.999 achieves the single highest result (0.946 in Adapts All), SARSA with decay 0.995 yields better overall consistency and higher average survivor rates: it outperforms or matches the 0.999 variant in Adapts Scout, Adapts Defender, and Adapts Support, where reliable generalisation across team structures matters more than peak performance in the all-adaptive setting. In a survival domain where death incurs a -20 penalty, SARSA’s on-policy, risk-averse updates during exploration prove slightly advantageous over Q-Learning’s more optimistic off-policy targets. The no-decay variant ($\epsilon=0.25$) also performs competitively, suggesting the compact state space (206–224 states) is small enough that partial exploration suffices.

Event Analysis: Table 4 shows that the fully adaptive team completes missions in fewer steps (68.7 vs. 94.2 baseline) and performs substantially more scans (98.1 vs. 70.5), confirming more aggressive and effective exploration. Heal events drop slightly (5.4 vs. 5.6), an acceptable trade-off given the much faster mission completion.

Table 4: Average events per episode (SARSA, decay 0.995).

Config	Steps	Attacks	Heals	Scans
Baseline	94.2	8.8	5.6	70.5
Adapts Scout	92.6	10.7	6.7	40.0
Adapts Defender	81.4	6.9	5.2	75.8
Adapts Support	90.9	8.3	2.6	80.9
Adapts (All)	68.7	9.9	5.4	98.1

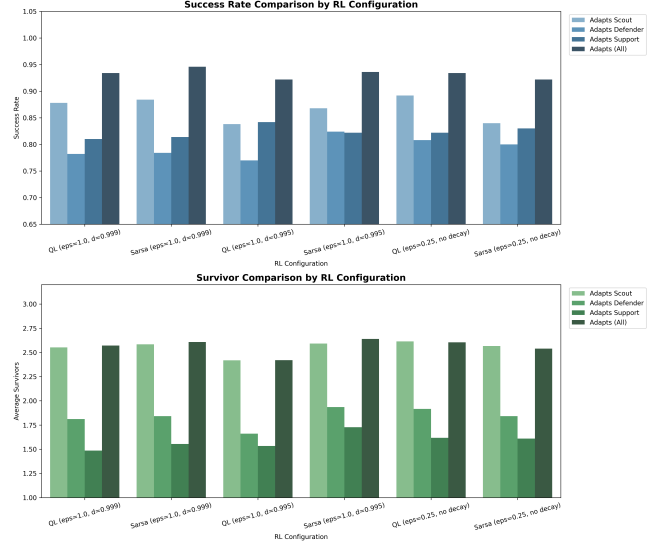


Figure 5: Test-phase comparison of Q-Learning and SARSA across all decay configurations and team modes.

Q-Table Analysis: To verify that the learned policy aligns with the shaped rewards, we extracted Q-values for the most confident states (Figure 6). In highly dangerous states (critical HP, adjacent zombies), the defender action commands the highest Q-value. Conversely, support dominates when a teammate is injured. States where the safe zone is unknown and there is no immediate danger reliably favour scout. This confirms the agent has learned human-interpretable, reward-consistent priorities rather than exploiting random regularities.

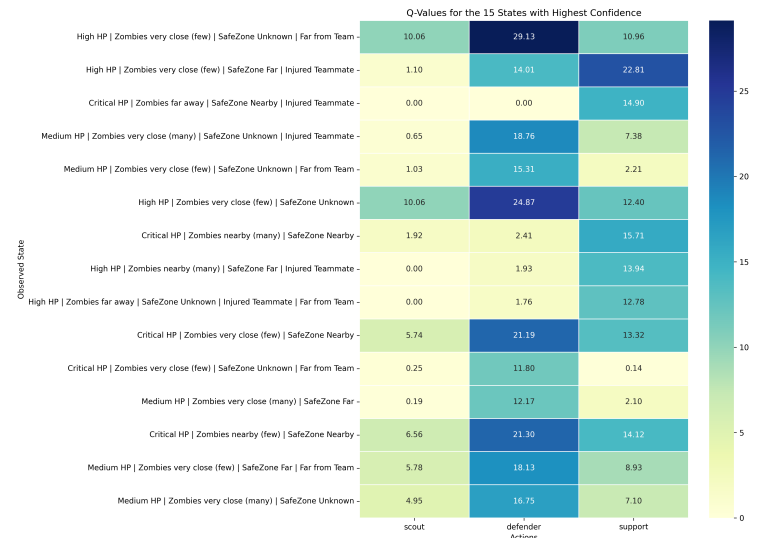


Figure 6: Q-values for the 15 most confident states.

6 Limitations and Future Work

The primary limitation is the use of tabular RL: although it works well for this compact state space (206–224 reachable states), it does not scale to richer observations or larger grids. A second limitation is that the adaptive agent selects among pre-built role behaviours rather than learning primitive actions such as move, attack, heal, and scan from scratch. This makes the approach more interpretable, but also bounds its expressiveness by the quality of the hand-coded roles.

The state representation is also intentionally compact: it omits relative teammate positions, individual teammate health, and action history, which limits the agent’s ability to adapt to mid-episode teammate loss or degraded team composition.

A limitation of the empirical evaluation is that results are based on a fixed grid size, fixed numbers of zombies and obstacles, and 500 test episodes per configuration. Although this is sufficient to compare the implemented alternatives, the conclusions may not generalise to larger maps, different zombie densities, or environments with different safe-zone placement rules.

Future work includes testing richer state representations with explicit teammate modelling, evaluating multiple environment sizes, obstacle densities, and random seeds, enabling inter-agent communication, and replacing role-selection with primitive-action Deep Q-Learning to reduce the dependency on hand-coded behaviours.

7 Conclusion

We presented a reproducible ad hoc teamwork benchmark and showed that a tabular RL agent can learn effective role-switching strategies without prior coordination. The adaptive agent improves over the hand-coded baseline in the Scout replacement and fully adaptive settings, while remaining competitive in the Defender and Support replacement settings.

Using the selected SARSA configuration with ϵ decay 0.995, the fully adaptive team achieved a 12.0 percentage-point improvement in success rate over the baseline while preserving an average of 2.64 survivors. These results suggest that learning a high-level role-selection policy is a practical and interpretable approach for ad hoc teamwork in small cooperative multi-agent environments.

8 Use of AI Tools

ChatGPT, Claude, and Gemini were used throughout the project for code generation, debugging, and help structuring written content. Interaction with these models occurred exclusively through their conversational web interfaces. Additionally, an IDE copilot was used strictly for minor syntax auto-completion and small formatting details.

All AI-generated code was reviewed and tested by the authors before integration. The core design decisions, state representation and reward function structure were made independently by the team.

References

- [1] P. Stone, G. Kaminka, S. Kraus, and J. Rosenschein. Ad hoc autonomous agent teams: Collaboration without pre-coordination. In *Proc. AAAI*, 2010.
- [2] D. Masad and J. Kazil. Mesa: An agent-based modeling framework. In *Proc. SciPy*, 2015.
- [3] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.
- [4] C. Claus and C. Boutilier. The dynamics of reinforcement learning in cooperative multiagent systems. In *Proc. AAAI*, 1998.
- [5] L. Panait and S. Luke. Cooperative multi-agent learning: The state of the art. *Autonomous Agents and Multi-Agent Systems*, 11:387–434, 2005.